SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	**Total Marks:**	100
CO4 Statement:	*Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.*		
Student Name:	Sree B	**Reg. No.:**	192524023
Section / Batch:	AI and DS / 2025	**Date:**	26/08/2026

Code of Conduct

I Sree B(1925240230 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sree B

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	100
	GRAND TOTAL:	**100 Marks**	

Annexure A – Comparative Analysis

1.Compute number of ways to achieve a sum using dice and compare with computing binomial coefficients. Analyze similarities in DP table construction. Identify differences in recurrence relations. Evaluate computational complexity. Discuss memory optimization. Generate insights on combinatorial DP problems.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

Sree B
192524023

QUESTIONS

Q1
Dice Throw Problem vs Binomial Coefficient (DP Comparison)
100 marks

0/1 answered

Q1 **Dice Throw Problem vs Binomial Coefficient (DP Comparison)** 100 marks

Compute number of ways to achieve a sum using dice and compare with computing binomial coefficients. Analyze similarities in DP table construction. Identify differences in recurrence relations. Evaluate computational complexity. Discuss memory optimization. Generate insights on combinatorial DP problems.

Your Code

```python
def dice_throw(num_dice, num_faces, target_sum):
    # dp[i][j] = number of ways to get sum j using i dice
    dp = [[0 for _ in range(target_sum + 1)] for _ in range(num_dice +
    dp[0][0] = 1

    for i in range(1, num_dice + 1):
        for j in range(1, target_sum + 1):
            for face in range(1, num_faces + 1):
                if j - face >= 0:
                    dp[i][j] += dp[i - 1][j - face]

    return dp[num_dice][target_sum]


def binomial_coefficient(n, k):
    dp = [[0 for _ in range(k + 1)] for _ in range(n + 1)]

    for i in range(n + 1):
```

Input

stdin input

Output

```
sum(dp[i-1][j-face]) for face in
1..num_faces
    (sums over multiple previous
states - a 'range' sum)
  - Binomial Coefficient: dp[i]
[j] = dp[i-1][j-1] + dp[i-1][j]
```